

Entrega de Desenvolvimento 2

Este documento mostra como as APIs REST em Java e Kotlin com Spring do projeto atendem cada requisito da disciplina, com link para o arquivo que comprova.

Período	Responsável	Versão	Classificação
2026	**Equipe do projeto**	**1.0 · 02/11/2026**	**Entrega acadêmica**

Sumário

Requisitos e evidências da disciplina

01

Visão geral 3

O que a disciplina pede e como ler este documento

02

Requisitos 4

Mínimos, extras e repositórios, com a evidência de cada um

01

As APIs REST com Spring atendem todos os requisitos mínimos e vários extras

O que a disciplina pede e o que o projeto entrega?

O que a disciplina pede

A disciplina pede uma API REST em Java com Spring MVC integrada a um banco SQL Postgres, com Spring Data JPA, CRUD, divisão em pacotes do padrão MVC, validação de entradas e tratamento centralizado de exceções. Os extras incluem documentação com Swagger, uso de procedures e functions do banco, autenticação e autorização com Spring Security e uma segunda API para um banco NoSQL.

O que foi entregue

O repositório api-core reúne a API REST principal, com Spring Boot, JPA e Postgres. Ela separa controller, service, repository, domain e dto, valida os corpos das requisições com Bean Validation e responde erros no formato Problem Details por um tratador central. O contrato OpenAPI é gerado e exportado, a segurança usa JWT e permissões por papel, e a procedure generate_subscription_charge é acionada pelo código Java. O repositório api-messenger traz uma segunda API Java, com MongoDB.

Como ler este documento

Cada requisito tem um identificador, o enunciado oficial, a descrição do que o projeto entrega e links para os arquivos ou pastas que o comprovam, todos na branch main do GitHub. Os identificadores que começam com M são os requisitos mínimos para a nota 7,0, e os que começam com E são os extras. Os identificadores seguem o enunciado oficial da disciplina. Quando o mesmo padrão aparece em mais de um repositório, o documento mostra um exemplo e cita os demais pelo nome.

02

O api-core cumpre os seis requisitos mínimos e as APIs vão além com OpenAPI, procedure, Spring Security e MongoDB

Como cada requisito é atendido e onde está a evidência?

Requisitos mínimos

Id	Requisito	O que o projeto entrega	Onde ver
M1	Criar uma API REST em Java com Spring MVC para um banco SQL Postgres.	O api-core usa Spring Boot WebMVC e o driver do PostgreSQL. Os controladores REST são organizados por área, como catálogo, empresa, unidade, pedido, proposta e profissional. O contrato da API está em um arquivo OpenAPI e uma coleção Bruno traz as requisições de cada endpoint.	pasta controller em api-core openapi.yaml em api-core
M2	Utilizar o Spring Data JPA para integrar a aplicação com o Postgres.	O api-core declara a dependência spring-boot-starter-data-jpa, tem repositórios JPA para o acesso aos dados e entidades mapeadas para as tabelas.	pasta repository em api-core pasta entity em api-core
M3	Implementar os métodos do CRUD no banco por meio da API.	A API cria, lista, edita e exclui unidades, contas de energia e fotos da unidade, entre outros recursos. Os serviços guardam as regras de cada operação.	LocalUnitController.java em api-core pasta service em api-core
M4	Aplicar a divisão de responsabilidades com os pacotes do padrão MVC.	O código separa os pacotes controller, service, repository, domain (entidades e enums) e dto (objetos de entrada e de saída). O documento ARCHITECTURE.md descreve as camadas.	ARCHITECTURE.md em api-core pasta service em api-core
M5	Validar as entradas recebidas pela API.	Os DTOs de entrada usam Bean Validation e os controladores aplicam @Valid nos corpos das requisições. O mesmo padrão aparece também em api-auth, nas rotas de autenticação.	pasta request em api-core LocalUnitController.java em api-core
M6	Tratamento centralizado de exceções, com respostas HTTP úteis para o cliente.	Uma classe @RestControllerAdvice trata as exceções em um só lugar e monta as respostas no formato Problem Details. Há uma exceção própria para regras de negócio, e o acesso negado devolve 403 no mesmo formato.	GlobalExceptionHandler.java em api-core ProblemDetailFactory.java em api-core

Requisitos extras

Id	Requisito	O que o projeto entrega	Onde ver
E1	Utilizar o Swagger para gerar documentação automática da API.	O api-core usa o springdoc-openapi, com anotações OpenAPI por área, e exporta o contrato em um arquivo openapi.yaml. O mesmo padrão aparece também em api-messenger.	openapi.yaml em api-core pasta openapi em api-core
E2	Acionar as Procedures e Functions criadas na disciplina Modelagem de Dados.	O api-core aciona a procedure generate_subscription_charge com CALL no SubscriptionRepository, e o endpoint de cobrança está documentado. A migração V1__baseline.sql define a procedure e as functions de trigger.	SubscriptionRepository.java em api-core V1__baseline.sql em api-core
E3	Implementar autenticação e autorização com Spring Security.	O api-core tem a cadeia de filtros e as regras de acesso do Spring Security, autorização por permissões (RBAC), chave de serviço nas rotas internas e validação do JWT emitido pelo api-auth. O mesmo padrão aparece também em api-auth.	SecurityConfig.java em api-core pasta rbac em api-core
E4	Criar mais uma API REST em Java com Spring MVC para um banco NoSQL.	O api-messenger é uma API REST em Java com Spring MVC e Spring Data MongoDB. Os controladores tratam conversas e mensagens e os documentos MongoDB ficam no pacote de modelo. O api-notify é uma terceira API Java com MongoDB, para notificações.	pasta controller em api-messenger pasta model em api-messenger

Repositórios da disciplina

Repositório	Papel na disciplina
api-core	API REST principal, com Spring MVC, Spring Data JPA e Postgres, e os requisitos mínimos.
api-auth	API de autenticação em Kotlin com Spring Security, que emite o JWT.
api-messenger	Segunda API REST em Java, com MongoDB, para conversas e mensagens.
api-notify	Terceira API Java com MongoDB, para notificações.



Entrega acadêmica.